VITyarthi | Open Source Software

The Open Source Audit

Capstone Project | OSS NGMC Course | Units 1–5

Software Under Audit

Linux Kernel

Student Name	Mangal Singh
Registration Number	24BCE10599
Course	Open Source Software	Units 1–5
Software	Linux Kernel (kernel.org)
License	GNU GPL v2
Environment	Ubuntu 24.04.4 LTS on WSL2
Date	30-03-2026

Table of Contents

Table of Contents 2
1. Introduction 3
Part A — Origin and Philosophy 4
A1. The Problem the Linux Kernel Was Created to Solve 4
A2. The License — What It Actually Says 5
A3. The Ethics of Open Source 6
Part B — Linux Kernel Footprint on a Linux System 7
B1. Installation 7
B2. Key Directories 7
B3. User, Group, and Security 7
B4. Key Commands 7
B5. Update Model 8
Part C — The FOSS Ecosystem 9
C1. Dependencies 9
C2. What Linux Has Enabled 9
C3. The LAMP Stack Connection 9
C4. Community and Governance 9
Part D — Open Source vs Proprietary: Critical Analysis 10
D1. Comparison Table 10
D2. Verdict 10
Part E — Shell Script Documentation 11
Script 1 — System Identity Report 11
Script 2 — FOSS Package Inspector 12
Script 3 — Disk and Permission Auditor 12
Script 4 — Log File Analyzer 14
Script 5 — Open Source Manifesto Generator 14
Conclusion 16
References 16

1. Introduction

When I started this course, my assumption was that open source was mainly about software being free to download. The more I researched, the more I realised that framing misses the point entirely. Open source is about who holds power over the tools people depend on — and whether that power sits with one corporation or is shared with everyone.

I chose the Linux Kernel because it's the clearest example of what open source can actually accomplish. A 21-year-old student in Finland wrote the first version in 1991 calling it a hobby project. Today that same kernel runs inside nearly every Android phone, all major cloud platforms — AWS, Google Cloud, Azure — and over 96% of the world's top 500 supercomputers. The distance between where it started and where it ended up is inseparable from the decision to make it open and to license it in a way that kept it that way.

This report covers the origin story, the license and what it actually means, the ethical questions open source raises, how the kernel lives on a real Linux system, its ecosystem, and a comparison with its closest proprietary equivalent. The five shell scripts I wrote and tested on Ubuntu 24.04.4 LTS are documented at the end.

Part A — Origin and Philosophy

A1. The Problem the Linux Kernel Was Created to Solve

The story of Linux doesn't really start with Linus Torvalds. It starts with the mess the Unix situation had become by the early 1990s.

The Unix Problem

Unix was developed at AT&T Bell Labs in the 1960s and by the 1980s was the operating system serious computing ran on. The problem was AT&T owned it. A proper Unix license cost tens of thousands of dollars, locking out students and researchers without institutional access. MINIX existed as a partial solution — Andrew Tanenbaum had written it as a teaching tool — but he deliberately kept it limited and wasn't interested in turning it into a production system. When users requested improvements on the comp.os.minix newsgroup, his response was consistently that MINIX was fine as it was.

Richard Stallman's Missing Piece

Since 1983, Richard Stallman had been building the GNU Project — a completely free Unix-like OS from scratch. By 1991 GNU had an excellent compiler (GCC), editor (Emacs), shell (Bash), and C library. But the kernel, called Hurd, was years away from working. GNU had everything except the one component that tied it all together.

> *"Hello everybody out there using minix — I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones." — Linus Torvalds, comp.os.minix, August 25, 1991*

Linus Torvalds and the GPL Decision

Linus Torvalds was a CS student at the University of Helsinki. He bought a 386 PC in 1991 and wanted a proper Unix-like environment on it. MINIX didn't do what he needed. So he started writing his own kernel — not to change the world, but to solve his own problem. He shared it anyway, and that decision is what made everything else possible.

The initial version had a restrictive license, but in 1992 Linus re-licensed the kernel under GPL v2. This was the crucial moment. The GPL's copyleft clause meant no one could ever take the kernel private — anyone who distributed a modified version had to release their changes under the same license. It also meant Linux could be legally combined with GNU's existing tools, giving the world a complete free operating system for the first time. Within two years there were proper Linux distributions. Within a decade, serious companies were deploying it in production.

Scale Today	The Linux Kernel now has over 27 million lines of code, contributions from 4,500+ developers and 500+ companies per cycle. Top corporate contributors include Intel, Google, Samsung, Red Hat, IBM, and Microsoft.

A2. The License — What It Actually Says

The Linux Kernel uses the GNU General Public License version 2 (GPL v2). The full text is in the COPYING file at kernel.org. I read through it — it's actually readable, unlike most legal documents.

The Four Freedoms

- Freedom 0 — Run the software for any purpose. No restrictions on use case whatsoever.
- Freedom 1 — Study how it works and change it. Requires source code access, which kernel.org provides fully.
- Freedom 2 — Redistribute copies. Canonical does this commercially with Ubuntu. Anyone can do the same.
- Freedom 3 — Distribute modified versions, but under the same license. This is the copyleft clause that keeps the ecosystem open.

GPL v2 grants all four. This isn't automatic — many licenses grant some but not others. MIT grants all four but doesn't enforce that modifications stay open, which is the critical difference.

Can Companies Modify Linux and Keep Changes Private?

It depends on distribution. The copyleft is triggered when you distribute modified software. If a company ships a product containing a modified kernel — like Android phones — they must release the kernel source. But if they only run a modified kernel on their own servers without distributing it externally, the GPL doesn't require them to release anything. This is how Google can run custom kernels on their infrastructure without publishing the changes.

GPL vs MIT

Dimension	GPL v2 vs MIT
Commercial use?	Both allow it
Must share modifications?	GPL: Yes if distributing. MIT: Never.
Can relicense as proprietary?	GPL: No. MIT: Yes.
Protects community?	GPL: Strongly via copyleft. MIT: Not at all.
Best for?	GPL: Foundational tools. MIT: Libraries, wide adoption.

For the kernel — where the goal is ensuring the foundation of computing stays free regardless of commercial pressure — GPL v2 is clearly right. If Linus had used MIT, companies could have taken the kernel private. The GPL made that legally impossible. For a personal utility where I want maximum adoption without barriers, I'd use MIT instead.

Free Beer vs Free Speech

'Free as in free beer' means zero cost. 'Free as in free speech' means freedom to use, study, modify, and share. Red Hat Enterprise Linux costs thousands of dollars per year — it's not free beer. But the GPL guarantees the source is available and your rights can't be removed — it's free speech. This is the distinction that makes GPL powerful: it protects freedom even when money changes hands. Proprietary apps that are free to download but have no source access flip this — free beer without free speech, which is the less important of the two.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The case for it is strong. Openly auditable software is fundamentally more trustworthy — no hidden backdoors, no secret data collection that can't be discovered. It also democratises access: a student anywhere can download GCC, the Linux Kernel, and Python and build competitive software at zero cost. Open source lowers barriers to participation in technology globally in ways that matter.

But the argument against is serious too. High-quality software requires sustained skilled effort. Many open-source maintainers are providing a public service without compensation — the Log4j situation in 2021 showed what this looks like at scale: one volunteer maintaining critical infrastructure embedded in millions of enterprise systems with no financial support. Some software also legitimately needs controlled release processes — cryptographic implementations, medical device firmware — where open publication before patches are ready causes direct harm. 'All software should be open source' is too simple a position.

Is It Ethical for Companies to Profit from Community Labor?

Companies like Google and Amazon save billions annually running on Linux. The case for it being fine: the GPL creates legal reciprocity. Companies must contribute kernel changes back, and the largest users — Google, Intel, IBM, Red Hat — are also the largest contributors. Google employs engineers specifically to work on the kernel. Intel maintains its own processor drivers. Corporate use and community benefit can align.

The more uncomfortable cases are with permissively licensed software. The Amazon vs. Elasticsearch situation showed what happens without legal reciprocity: AWS built a multi-billion-dollar managed service on community software and contributed essentially nothing back. The GPL specifically prevents this for the kernel. The broader ecosystem, full of MIT and Apache-licensed tools, has a real exploitation problem that copyleft would have prevented.

Standing on the Shoulders of Giants

Every program I've written ran on an OS kernel, compiled by a compiler, linked against libraries — almost all of it open source. Open source dramatically lowers the floor of what you need to do before you can start innovating. A two-person startup can compete on the same foundational infrastructure as a billion-dollar company. Far from undermining innovation, open source accelerates it by pre-solving foundational problems so people can focus on what's actually new and different.

Part B — Linux Kernel Footprint on a Linux System

All commands below were run on my Ubuntu 24.04.4 LTS system (WSL2, kernel 6.6.87.2-microsoft-standard-WSL2). Outputs are real.

B1. Installation

On Ubuntu and Debian-based systems, the kernel arrives through apt as three packages: linux-image-[version] (the compiled binary), linux-headers-[version] (headers for building modules), and linux-modules-[version] (loadable drivers). On RPM systems like Fedora, the equivalent comes through dnf. For custom kernels, the source can be compiled from kernel.org — this requires GCC, make, and dependencies, taking 30-90 minutes on modern hardware.

B2. Key Directories

Path	Purpose
/boot/vmlinuz-*	The compiled kernel binary loaded by GRUB at startup
/boot/initrd.img-*	Initial RAM disk — temporary filesystem for early boot
/lib/modules/$(uname -r)/	Loadable kernel modules (device drivers, filesystem drivers)
/proc/	Virtual filesystem — live kernel data: processes, memory, CPU info. Zero disk space.
/sys/	Virtual filesystem — hardware topology and driver info exposed by kernel
/etc/sysctl.conf	Tunable kernel parameters: networking, memory, security settings
/var/log/kern.log	Kernel log messages: hardware events, driver errors, warnings

B3. User, Group, and Security

The kernel runs in ring 0 (highest CPU privilege) as root (UID 0) — it defines what root means for everything else. User programs run in ring 3 and cannot touch hardware directly. They must go through system calls, which the kernel validates. Kernel modules run in kernel space with full privileges, which is why modern distributions require digitally signed modules and Secure Boot enforces this at startup.

B4. Key Commands

```
uname -r                          # running kernel version
cat /proc/version                 # full kernel build info
cat /proc/meminfo | head -5       # live memory stats from kernel
lsmod | head -10                  # currently loaded kernel modules
dpkg -l | grep linux-image        # installed kernel packages
sudo modprobe <module>            # load a module without rebooting
sudo apt update && sudo apt upgrade  # update kernel via package manager
```

B5. Update Model

Linus Torvalds releases a new stable kernel roughly every 9-10 weeks, with security point releases much faster — often within days of a confirmed vulnerability. Ubuntu's security team backports critical fixes to the kernel version shipped with the Ubuntu release. Running 'sudo apt upgrade' delivers these patches automatically, meaning a production server gets security fixes without needing a full kernel version upgrade.

Part C — The FOSS Ecosystem

C1. Dependencies

Building the Linux Kernel from source requires: GCC (or Clang/LLVM since v5.12) to compile the C source, GNU Make to manage the build process across thousands of interdependent files, GNU Binutils to link compiled object files into the kernel image, and OpenSSL to generate cryptographic signing keys for module signing. These are all major open-source projects in their own right.

C2. What Linux Has Enabled

- Android — Every Android phone runs the Linux Kernel. Google maintains a separate Android kernel tree with mobile-specific additions, but the base is Linux. Over 3 billion active devices.
- Linux Distributions — Ubuntu, Fedora, Debian, Arch, Kali, and hundreds of others are all built around the Linux Kernel. Each packages it differently but the kernel is constant.
- Containers — Docker, Kubernetes, and the container ecosystem are built on Linux Kernel namespaces (process isolation) and cgroups (resource limiting). Without these kernel features, modern cloud-native computing wouldn't exist.
- Cloud Computing — AWS, Google Cloud, Azure all run Linux. The economics of cloud computing depend on the OS being free. Running a million VMs on proprietary OS licenses would fundamentally change cloud pricing.
- Supercomputers — 100% of the top 500 supercomputers run Linux, consistently across recent Top500 lists.

C3. The LAMP Stack Connection

The L in LAMP (Linux, Apache, MySQL, PHP) is the Linux Kernel. WordPress, which powers ~43% of all websites, runs on LAMP. The kernel manages the network connections Apache serves, schedules CPU time between processes, and handles the disk I/O MySQL uses. It's the invisible layer underneath every website running on this stack.

C4. Community and Governance

Development happens through the Linux Kernel Mailing List (lkml.org), handling thousands of messages per day. Contributions arrive as email patches, are reviewed publicly, debated, tested, and merged or rejected. Governance is a meritocracy — Linus Torvalds at the top, with a hierarchy of subsystem maintainers who earned authority through quality contributions over time. The Linux Foundation provides administrative and financial support, manages kernel.org infrastructure, and organises the annual Linux Kernel Summit. Its corporate members include IBM, Intel, Google, Samsung, and Microsoft.

Part D — Open Source vs Proprietary: Critical Analysis

The most direct comparison is Linux Kernel vs Windows NT Kernel — the two most widely deployed kernels in the world.

D1. Comparison Table

Dimension	Linux Kernel (Open Source)	Windows NT Kernel (Proprietary)
Cost	Free. No licensing fees at any scale.	Embedded in Windows license. $139–$199 retail, thousands for enterprise servers.
Security — Audit	Source public at kernel.org. Any researcher can audit it.	Source private. Only Microsoft and partners can audit.
Support	Community + commercial (Red Hat, Canonical). Powers all cloud and HPC.	Official Microsoft SLAs. Strong for enterprise. More expensive at scale.
Freedom to Modify	Full. Download, change, recompile, deploy. GPL keeps changes open.	None. Source unavailable to end users.
Control	Meritocratic. Linus + maintainer hierarchy. No single corporation.	100% Microsoft. All decisions driven by one company's shareholder interests.
Verdict	Server, cloud, embedded, HPC — clear choice.	Consumer desktop, gaming, Microsoft-ecosystem environments.

D2. Verdict

For server and cloud deployments, the Linux Kernel wins — not by default but on genuine merit. Zero cost, full auditability, thirty-plus years of optimisation by thousands of contributors, and the flexibility to customise for specific workloads is something Windows simply can't match. The evidence is that essentially all cloud infrastructure already runs on it.

Windows NT genuinely wins on consumer desktop and gaming. Microsoft's DirectX and decades of consumer hardware driver support give it advantages Linux is only recently closing, partly through Valve's Proton layer and the Steam Deck. In most enterprises the realistic answer is both: Linux for servers, Windows for desktops. I'd contribute to the kernel — starting with documentation and small bug fixes in drivers for hardware I own, working toward understanding a subsystem well enough to submit real patches.

Part E — Shell Script Documentation

All five scripts were written and tested on Ubuntu 24.04.4 LTS (WSL2, kernel 6.6.87.2-microsoft-standard-WSL2). Full script files are in the GitHub repository. Below are the key code sections demonstrating the required concepts, with explanations.

Script 1 — System Identity Report

File	script1_system_identity.sh

Displays a formatted welcome screen showing the Linux distribution, kernel version, architecture, logged-in user, home directory, system uptime, date/time, and the GPL v2 license message. Demonstrates variables, command substitution, and echo formatting.

Key Concepts: Variables and Command Substitution

```
#!/bin/bash
# Script 1: System Identity Report — Author: Mangal | 24BCE10599

STUDENT_NAME="Mangal"                # variable storing a fixed string
LICENSE="GNU General Public License v2 (GPL v2)"

# command substitution $() — runs each command and stores its output
KERNEL=$(uname -r)                   # current kernel version
DISTRO=$(lsb_release -d 2>/dev/null | cut -f2)  # distro name
USER_NAME=$(whoami)                  # logged-in user
UPTIME=$(uptime -p)                  # human-readable uptime
CURRENT_DATE=$(date '+%A, %d %B %Y %H:%M')     # formatted date

echo "  Kernel Ver   : $KERNEL"
echo "  OS License   : $LICENSE"
```

Full script available at: github.com/livingmangal/oss-audit-24BCE10599

SCREENSHOT —

```
mangal@DESKTOP-V4KK6J7:~/oss-audit$ cd ~/oss-audit
mangal@DESKTOP-V4KK6J7:~/oss-audit$ ./script1_system_identity.sh
==============================================
   Open Source Audit — Mangal
   Software Under Audit: Linux Kernel
==============================================

  Distribution : Ubuntu 24.04.4 LTS
  Kernel Ver   : 6.6.87.2-microsoft-standard-WSL2
  Architecture : x86_64
  User         : mangal
  Home Dir     : /home/mangal
  Uptime       : up 15 minutes
  Date & Time  : Sunday, 29 March 2026 17:30

----------------------------------------------
  OS License   : GNU General Public License v2 (GPL v2)

  The Linux Kernel is covered by GPL v2.
  This means anyone can view, modify, and
  redistribute the source code — as long as
  they share their changes under the same
  license. Freedom is protected by law.
----------------------------------------------
```

Script 2 — FOSS Package Inspector

File	script2_package_inspector.sh

Checks whether the Linux kernel image package is installed using dpkg, shows version info, and uses a case statement to print a philosophy note. On WSL2, the kernel is Microsoft-managed so dpkg falls back to uname output — the script handles this gracefully.

Key Concepts: if-then-else, case statement, dpkg, pipe and grep

```
#!/bin/bash
# Script 2: FOSS Package Inspector — Author: Mangal | 24BCE10599

PACKAGE="linux-image-$(uname -r)"

# if-then-else: check if package exists in dpkg database
if dpkg -l "$PACKAGE" &>/dev/null; then
    dpkg -l "$PACKAGE" | grep -E "^ii" | awk '{ print "Version: "$3 }'
else
    echo "  Kernel Version : $(uname -r)"   # fallback for WSL2
fi

# case statement: match software name, print philosophy note
case $SOFTWARE in
    linux-kernel) echo "Linux Kernel: the open-source core of modern computing" ;;
    httpd)        echo "Apache: the web server that built the open internet" ;;
esac
```

Full script available at: github.com/livingmangal/oss-audit-24BCE10599

SCREENSHOT —

```
mangal@DESKTOP-V4KK6J7:~/oss-audit$ ./script2_package_inspector.sh
==============================================
  FOSS Package Inspector
==============================================

  [INFO] Exact package linux-image-6.6.87.2-microsoft-standard-WSL2 not found via dpkg.
  Showing general kernel info instead:

  Kernel Version : 6.6.87.2-microsoft-standard-WSL2
  Kernel Release : #1 SMP PREEMPT_DYNAMIC Thu Jun  5 18:30:46 UTC 2025

----------------------------------------------
  License & Source Info:
----------------------------------------------
  Linux Kernel: The open-source OS core that
  powers 96% of the world's top servers,
  Android phones, and every major cloud platform.
  Built by a student in 1991 — shared with the world.

  GPL v2 License: Source code must always remain free.
==============================================
```

Script 3 — Disk and Permission Auditor

File	script3_disk_auditor.sh

Uses a for loop to iterate through key system directories, reporting permissions, owner, group, and disk usage for each. A second loop audits kernel-specific directories (/proc, /sys, /lib/modules) with explanations of their purpose.

Key Concepts: array, for loop, if [-d], ls -ld, awk, du

```
#!/bin/bash
# Script 3: Disk and Permission Auditor — Author: Mangal | 24BCE10599

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot" "/proc" "/sys")

# for loop: iterate over every directory in the array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then   # if-then: test directory exists
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')  # extract permissions
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')  # extract owner
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1) # disk usage
        echo "$DIR => Perms: $PERMS | Owner: $OWNER | Size: $SIZE"
    fi
done
```

Full script available at: github.com/livingmangal/oss-audit-24BCE10599

SCREENSHOT —

```
mangal@DESKTOP-V4KK6J7:~/oss-audit$ ./script3_disk_auditor.sh
==================================================
  Directory Audit Report
  System: 6.6.87.2-microsoft-standard-WSL2
  Date  : 29 March 2026 17:32
==================================================

  Directory : /etc
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: root
  Size      : 4.3M
  --------------------------------------------------
  Directory : /var/log
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: syslog
  Size      : 17M
  --------------------------------------------------
  Directory : /home
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: root
  Size      : 48K
  --------------------------------------------------
  Directory : /usr/bin
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: root
  Size      : 95M
  --------------------------------------------------
  Directory : /tmp
  Perms     : drwxrwxrwt  |  Owner: root  |  Group: root
  Size      : 24K
  --------------------------------------------------
  Directory : /boot
  Perms     : drwxr-xr-x  |  Owner: root  |  Group: root
  Size      : 4.0K
  --------------------------------------------------
  Directory : /proc
  Perms     : dr-xr-xr-x  |  Owner: root  |  Group: root
  Size      : 0
  --------------------------------------------------
  Directory : /sys
  Perms     : dr-xr-xr-x  |  Owner: root  |  Group: root
  Size      : 0
  --------------------------------------------------

  === Linux Kernel Specific Directories ===

  [KERNEL DIR] /proc
  Perms: dr-xr-xr-x  |  Owner: root
  Purpose: Live kernel & process info (virtual, no disk space used)
  --------------------------------------------------
  [KERNEL DIR] /sys
  Perms: dr-xr-xr-x  |  Owner: root
  Purpose: Hardware & driver info exposed by kernel (virtual)
  --------------------------------------------------
  [KERNEL DIR] /lib/modules/6.6.87.2-microsoft-standard-WSL2
  Perms: drwxr-xr-x  |  Owner: root
  Purpose: Kernel modules (drivers) for current kernel version
  --------------------------------------------------

  Audit complete.
==================================================
```

Script 4 — Log File Analyzer

File	script4_log_analyzer.sh	Run: ./script4_log_analyzer.sh /var/log/dpkg.log install

Accepts a log file and keyword as command-line arguments, reads the file line by line counting keyword matches, then prints the last 5 matching lines. Includes error handling for missing arguments, missing files, and empty files with a retry prompt.

Key Concepts: $1/$2 arguments, while read loop, if-then, counter, grep | tail

```
#!/bin/bash
# Script 4: Log File Analyzer — Author: Mangal | 24BCE10599
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]

LOGFILE=$1              # command-line argument 1: log file path
KEYWORD=${2:-"error"}   # argument 2 with default value 'error'
COUNT=0                 # counter variable, starts at zero

# while read loop: process the file one line at a time
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then  # case-insensitive search
        COUNT=$((COUNT + 1))   # arithmetic expansion to increment counter
    fi
done < "$LOGFILE"
echo "'$KEYWORD' found $COUNT time(s)."
grep -i "$KEYWORD" "$LOGFILE" | tail -5  # last 5 matching lines
```

Full script available at: github.com/livingmangal/oss-audit-24BCE10599

SCREENSHOT —

```
mangal@DESKTOP-V4KK6J7:~/oss-audit$ ./script4_log_analyzer.sh /var/log/dpkg.log install
==============================================
  Log File Analyzer
  Kernel: 6.6.87.2-microsoft-standard-WSL2
==============================================

  Scanning : /var/log/dpkg.log
  Keyword  : 'install'

----------------------------------------------
  RESULT: 'install' found 2029 time(s) in
  dpkg.log
----------------------------------------------

  Last 5 lines containing 'install':

    >> 2026-03-29 17:23:33 status installed man-db:amd64 2.12.0-4build2
    >> 2026-03-29 17:23:33 status installed dbus:amd64 1.14.10-4ubuntu4.1
    >> 2026-03-29 17:23:33 trigproc install-info:amd64 7.1-3build2 <none>
    >> 2026-03-29 17:23:33 status half-configured install-info:amd64 7.1-3build2
    >> 2026-03-29 17:23:33 status installed install-info:amd64 7.1-3build2

  Analysis complete.
==============================================
```

Script 5 — Open Source Manifesto Generator

File	script5_manifesto.sh

Asks the user three interactive questions and builds a personalised open-source philosophy statement from their answers, saving it to a file named after their username. The manifesto generated during testing is in the repository as manifesto_mangal.txt.

Key Concepts: read -p, string concatenation, > and >> file redirection, date, alias concept

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator — Author: Mangal | 24BCE10599
# alias concept: alias today='date +%d\ %B\ %Y' — shown here as a variable instead

DATE=$(date '+%d %B %Y')          # current date via command substitution
OUTPUT="manifesto_$(whoami).txt"  # filename built from username

# read -p: interactive prompts that capture user input
read -p "  1. One open-source tool you use every day: " TOOL
read -p "  2. In one word, what does freedom mean to you? " FREEDOM
read -p "  3. One thing you would build and share freely: " BUILD

# string concatenation and file writing with > (create) and >> (append)
echo "MY OPEN SOURCE MANIFESTO | $(whoami) | $DATE" > "$OUTPUT"
echo "Every day I use $TOOL — shared freely by someone before me." >> "$OUTPUT"
echo "Like Linus in 1991, I want to build $BUILD for everyone." >> "$OUTPUT"
cat "$OUTPUT"   # print the saved file to screen
```

Full script available at: github.com/livingmangal/oss-audit-24BCE10599

SCREENSHOT —

```
mangal@DESKTOP-V4KK6J7:~/oss-audit$ ./script5_manifesto.sh
==============================================
  Open Source Manifesto Generator
  Linux Kernel Audit Project
==============================================

  Answer three questions to create your
  personal open source philosophy statement.

  1. Name one open-source tool you use every day: Linux
  2. In one word, what does 'freedom' mean to you? Our decisions not influenced by others
  3. Name one thing you would build and share freely: IOT helping in Environment Sustainability

  Generating your manifesto...

  Manifesto saved to: manifesto_mangal.txt

==============================================
  MY OPEN SOURCE MANIFESTO
  By: mangal | Date: 29 March 2026
==============================================

  Every day, I rely on Linux — a tool that
  exists because someone chose to build in
  the open and share their work freely.

  To me, freedom means Our decisions not influenced by others. That is
  exactly what the Linux Kernel represents:
  the freedom to see how your computer works,
  change it, and pass that knowledge on.

  Inspired by Linus Torvalds — who shared his
  kernel with the world in 1991 — I too want
  to build IOT helping in Environment Sustainability and release it freely,
  so that anyone, anywhere, can use it,
  improve it, and stand on my shoulders
  just as I stand on the shoulders of those
  who came before me.

  This is the open source promise.
==============================================
```

Conclusion

What I came away from this project with is a better sense of why open source is the way it is — not just what it is. The kernel exists because someone was frustrated by proprietary limitations, shared their solution, and chose a license that legally prevented anyone from ever making it private again. The GPL v2 isn't just a preference — it's an enforcement mechanism for a philosophical position. Linus could have used a permissive license and the kernel might still have succeeded. But it could also have been captured. The GPL made that impossible, and that irreversibility is why the ecosystem around Linux has grown the way it has.

The ethical questions don't resolve cleanly, and I think that's the honest answer. Whether all software should be open source, whether corporate use of community work is always ethical, how to think about innovation that builds on freely shared foundations — these are real tensions without tidy resolutions. The technical side grounded all of this: exploring /proc, /sys, and /boot, watching the kernel expose live hardware information through a filesystem that takes zero disk space, understanding how security patches flow from Linus's tree to a 'sudo apt upgrade' on my machine. The shell scripts connected it further — writing them forced me to understand the tools rather than just run them. The Linux Kernel is not just a free product. It's a demonstration that a particular way of organising collective human effort can produce something no single organisation acting alone could have built.

References

All written sections are in my own words. These sources were used for research and fact-checking:

- Linus Torvalds, original August 25, 1991 post — comp.os.minix (archived on Google Groups)
- GNU Project — The Free Software Definition: gnu.org/philosophy/free-sw.html
- Open Source Initiative — The Open Source Definition: opensource.org/osd
- Eric S. Raymond — The Cathedral and the Bazaar: catb.org/~esr/writings/cathedral-bazaar
- Linux Kernel source and GPL v2 license text (COPYING file): kernel.org
- SPDX License List — GPL-2.0-only: spdx.org/licenses/GPL-2.0-only.html
- The Linux Command Line by William Shotts: linuxcommand.org
- GNU Bash Reference Manual: gnu.org/software/bash/manual
- Linux Kernel Mailing List: lkml.org
- Linux Foundation: linuxfoundation.org/about
- Torvalds, Linus — Just for Fun: The Story of an Accidental Revolutionary (2001)
- Ubuntu Security Notices: ubuntu.com/security/notices